Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19/03/2026**	**Enrolment No:92301733054**

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models. Take the words —-> Give their numeric representation —-> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as “I”, “a” may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus
2. Perform TF vectorization of the corpus

Program (Code):

```
sentences = [
   "India is known for its unity in diversity.",
   "Many tourists visit India to explore its culture and history.",
   "India has beautiful mountains, rivers, and beaches.",
   "Basketball is an exciting and fast-paced game.",
   "Playing games improves focus and teamwork skills.",
   "Technology makes communication easier across the world.",
   "Artificial Intelligence helps solve complex problems quickly."
]
#IMPORTING BASIC LIBRAIES
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer#TF-BOW
from sklearn.feature_extraction.text import TfidfVectorizer#TF-IDF
from sklearn.metrics.pairwise import cosine_similarity
#TF-BOW Embedding
tf_vectorizer=CountVectorizer()
tf_emb=tf_vectorizer.fit_transform(sentences)
#similarity matrix using cosine_similarity
tf_sim=cosine_similarity(tf_emb)
#TF-IDF Embedding
tridf_vectorizer=TfidfVectorizer()
tridf_emb=tridf_vectorizer.fit_transform(sentences)
tfidf_sim=cosine_similarity(tridf_emb)
#Recommend the most matching/similar sentences
def doc_sim(tfidf_sim,sentence):
  for i in range(len(sentences)):
    sim=tfidf_sim[i].copy()
    sim[i]=-1
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19/03/2026**	**Enrolment No:92301733054**

```
top_index=np.argmax(sim)
print(sentences[i],"--->",sentences[top_index])
```

Results:

```
tf_sim

array([[1.        , 0.77777778, 0.58925565, 0.27216553, 0.09245003,
        0.        , 0.        , 0.12598816, 0.11785113],
       [0.77777778, 1.        , 0.47140452, 0.27216553, 0.09245003,
        0.        , 0.        , 0.12598816, 0.11785113],
       [0.58925565, 0.47140452, 1.        , 0.14433757, 0.19611614,
        0.        , 0.        , 0.26726124, 0.125     ],
       [0.27216553, 0.27216553, 0.14433757, 1.        , 0.33968311,
        0.16666667, 0.        , 0.15430335, 0.        ],
       [0.09245003, 0.09245003, 0.19611614, 0.33968311, 1.        ,
        0.1132277 , 0.        , 0.20965697, 0.        ],
       [0.        , 0.        , 0.        , 0.16666667, 0.1132277 ,
        1.        , 0.5       , 0.15430335, 0.28867513],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.5       , 1.        , 0.3086067 , 0.14433757],
       [0.12598816, 0.12598816, 0.26726124, 0.15430335, 0.20965697,
        0.15430335, 0.3086067 , 1.        , 0.13363062],
       [0.11785113, 0.11785113, 0.125     , 0.        , 0.        ,
        0.28867513, 0.14433757, 0.13363062, 1.        ]])
```

```
tfidf_sim

array([[1.        , 0.15526276, 0.07912306, 0.09879606, 0.        ,
        0.        , 0.        ],
       [0.15526276, 1.        , 0.12433161, 0.04875111, 0.0513164 ,
        0.        , 0.        ],
       [0.07912306, 0.12433161, 1.        , 0.05884753, 0.0619441 ,
        0.        , 0.        ],
       [0.09879606, 0.04875111, 0.05884753, 1.        , 0.05651085,
        0.        , 0.        ],
       [0.        , 0.0513164 , 0.0619441 , 0.05651085, 1.        ,
        0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        1.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 1.        ]])
```

```
doc_sim(tfidf_sim,sentences)

India is known for its unity in diversity. ---> Many tourists visit India to explore its culture and history.
Many tourists visit India to explore its culture and history. ---> India is known for its unity in diversity.
India has beautiful mountains, rivers, and beaches. ---> Many tourists visit India to explore its culture and history.
Basketball is an exciting and fast-paced game. ---> India is known for its unity in diversity.
Playing games improves focus and teamwork skills. ---> India has beautiful mountains, rivers, and beaches.
Technology makes communication easier across the world. ---> India is known for its unity in diversity.
Artificial Intelligence helps solve complex problems quickly. ---> India is known for its unity in diversity.
```

Observation:

- Bag of Words (BoW) converts text into numerical form based on word frequency
- TF (Term Frequency) gives importance to words based on how often they appear in a document
- TF-IDF improves BoW by reducing the weight of frequent but less important words
- Cosine similarity is used to measure similarity between documents
- TF-IDF gives better similarity results compared to BoW and TF
- Word embedding helps reduce dimensionality and capture meaningful relationships between words
- Similar sentences are correctly identified based on content similarity

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19/03/2026**	**Enrolment No:92301733054**

Post Lab Exercise:

Take three documents and find similarity using

a. BoW vectorization
b. TF Vectorization
c. TF-IDF Vectorization

Comment over the answer obtained in each of the case.

Program Code:

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
docs = [
    "Machine learning is very powerful",
    "Machine learning is widely used",
    "Football is a popular sport"
]
bow = CountVectorizer()
bow_vec = bow.fit_transform(docs)
bow_sim = cosine_similarity(bow_vec)
print("BoW Similarity:\n", bow_sim)
tf_sim = cosine_similarity(bow_vec)
tfidf = TfidfVectorizer()
tfidf_vec = tfidf.fit_transform(docs)
tfidf_sim = cosine_similarity(tfidf_vec)
print("\nTF-IDF Similarity:\n", tfidf_sim)
```

Result:

```
bow = CountVectorizer()
bow_vec = bow.fit_transform(docs)
bow_sim = cosine_similarity(bow_vec)

print("BoW Similarity:\n", bow_sim)

BoW Similarity:
 [[1.        0.6       0.2236068]
 [0.6       1.        0.2236068]
 [0.2236068 0.2236068 1.       ]]

tf_sim = cosine_similarity(bow_vec)
```

```
tfidf = TfidfVectorizer()
tfidf_vec = tfidf.fit_transform(docs)
tfidf_sim = cosine_similarity(tfidf_vec)

print("\nTF-IDF Similarity:\n", tfidf_sim)

...
TF-IDF Similarity:
 [[1.         0.42948853 0.10180788]
 [0.42948853 1.         0.10180788]
 [0.10180788 0.10180788 1.        ]]
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:19/03/2026**	**Enrolment No:92301733054**

Observation:

a. BoW Vectorization

- Based only on word count
- Does not consider importance of words
- Gives basic similarity results
- May give misleading similarity

b. TF Vectorization

- Same as BoW (frequency-based)
- No weighting of rare/common words
- Limited performance

c. TF-IDF Vectorization

- Considers both frequency and importance
- Reduces weight of common words (like “is”)
- Gives more accurate similarity results
- Best among all three methods